



16. Recurrent Neural Networks

Remember the past, understand the present, predict the future.

Previous Section:

[15. Neural Networks for Images](#)

Contents:

[1. What is a Recurrent Neural Network?](#)

[2. Implementation of Recurrent Neural Networks](#)

[2.1. Load and Explore the Data](#)

[2.2. Prepare the Data](#)

[2.3. Visualizing the Data](#)

[2.4. Building the Complete Preprocessing Pipeline](#)

[2.5. Sliding Window](#)

[2.6 Training a Recurrent Neural Network](#)

[Summary](#)

[References](#)

For the last three sections, we have explored Deep Learning primarily through the lens of Computer Vision. We taught machines how to understand images, recognize objects, classify scenes, and even segment individual pixels. In other words, our focus was on data where each example could be treated independently.

Now, we move into a different world. Imagine you are given daily stock prices from **2013-01-01 to 2017-08-15**. At first glance, it may seem like just another dataset. But there is one important difference: **the order matters**.

What this means is that: The value on January 2nd depends on what happened on January 1st. The value on January 3rd depends on both previous days. If we randomly shuffle the data, much of the information disappears. This is what we call **sequential data**.

Back in Machine Learning, we discussed how traditional supervised and unsupervised learning often assume that observations are independent from one another. Time series,

speech, text, and many real-world signals violate that assumption. Their meaning comes not only from the current observation but also from what happened before.

Because of this, sequential modeling becomes a unique challenge. Some common tasks involving sequential data include:

- **Classification:** Is this website visitor a human or a bot?
- **Regression:** What will next month's sales be?
- **Event Detection:** Did a specific event occur within the sequence?
- **Anomaly Detection:** Is there unusual behavior hidden in the data?

To solve these problems, we need a model capable of remembering information across time. And that is exactly where Recurrent Neural Networks enter the picture.

1. What is a Recurrent Neural Network?

A **Recurrent Neural Network (RNN)** is a neural network architecture specifically designed to process sequential data such as text, speech, sensor readings, and time series.

Unlike traditional neural networks, RNNs are built around a simple but powerful idea: ***The past should influence how we interpret the present.***

Think about translating a sentence from English into Vietnamese. Suppose you encounter the word: ***"bank"***. What does it mean? Is it a financial institution? Or is it the side of a river?

The answer depends entirely on the words that came before it.

- If the sentence is: *"I deposited money into the bank."* → The meaning is obvious.
- But if the sentence is: *"The fisherman sat near the bank."* → The meaning changes completely.

Humans naturally use previous context when interpreting language. We rarely process words in isolation. Instead, we continuously remember what has already been said and combine that information with the current word. RNNs attempt to mimic this behavior.

As an RNN processes a sequence, it maintains an internal representation known as a **hidden state**. This hidden state acts like a memory that carries information from earlier time steps.

When a new input arrives, the current input is processed. The previous hidden state is retrieved. Both pieces of information are combined. A new hidden state is generated.

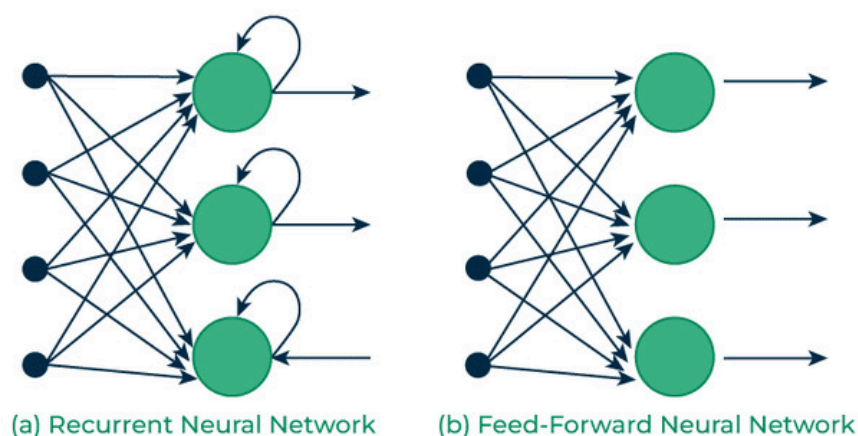
As a result, the model does not see individual observations as isolated events. Instead, it continuously builds an understanding of the sequence as it progresses. This allows the network to capture contextual relationships and temporal dependencies that would otherwise be lost.

For example:

- In text generation, the next word depends on previous words.
- In speech recognition, the current sound depends on earlier sounds.
- In stock prediction, today's value depends on previous market behavior.
- In weather forecasting, future conditions depend on historical patterns.

In all of these cases, remembering the past is essential.

To appreciate why RNNs are different, let us compare them with the neural networks we have used so far. As you may have known (or may not have known), traditional neural networks are more formally called **Feed-Forward Neural Networks**.



In a feed-forward network, information flows in one direction. Data enters through the input layer. It passes through one or more hidden layers. A prediction is produced at the output layer. And the computation ends.

Each sample is processed independently. If you show the network two sentences:

1. "I love machine learning."
2. "Deep learning is fascinating."

The network treats them as completely unrelated inputs. It has no mechanism for remembering the first sentence while processing the second.

For image classification, this is usually acceptable because each image is generally independent of every other image. In contrast, **sequential data is different**. When processing a sentence, a speech signal, or a time series, previous observations often contain crucial information needed to understand the current one.

This is where Recurrent Neural Networks introduce a fundamental architectural change. Inside an RNN, the hidden layer contains a **recurrent connection**. Instead of discarding information after each computation, the hidden state from the current step is passed forward to the next step.

Conceptually, the network performs something like this:

$$h_t = f(x_t, h_{t-1})$$

where: x_t is the current input, h_{t-1} is the previous hidden state, h_t is the new hidden state.

Notice what happened. The network is no longer relying solely on the current input. It is also using information accumulated from previous steps.

This recurrent connection creates a feedback loop that enables the network to maintain memory over time. As the sequence progresses, the hidden state gradually accumulates contextual information from earlier observations. Each new input updates that memory, allowing the model to continuously refine its understanding of the sequence.

This simple idea is what makes RNNs powerful. Rather than treating data points independently, they treat them as parts of an unfolding story. Every new observation becomes meaningful not only because of what it is, but also because of everything that came before it.

And that ability to remember context is what made Recurrent Neural Networks one of the first major breakthroughs in sequential deep learning.

2. Implementation of Recurrent Neural Networks

Now that we understand what a Recurrent Neural Network is and why sequential data requires memory, it is time to build one ourselves.

For this laboratory, we will use the famous **Store Sales Forecasting Dataset**. The goal is straightforward: ***Given historical information about stores, products, and promotions, can a model learn patterns from the past and estimate future sales?***

Google Drive link:

https://drive.google.com/drive/folders/1gBb6NTi62HUMKgeJgVKmHYUdmerY4TmF?usp=drive_link

Original Dataset: <https://www.kaggle.com/competitions/store-sales-time-series-forecasting/data>

```
file_path = "train.csv"
df = pd.read_csv(file_path).set_index('id')

print("Number of samples:", len(df))
```

Number of samples: 3000888

The original training dataset contains more than **3 million records**.

2.1. Load and Explore the Data

The Store Sales dataset is actually composed of several files. Each file contains a different piece of information that can later be merged together to create a richer forecasting model.

The available datasets are:

- `train.csv` — Historical sales records used for training
- `stores.csv` — Store metadata such as city, state, store type, and cluster
- `oil.csv` — Daily oil prices
- `holidays_events.csv` — Holiday and special event information
- `test.csv` — Dataset used for forecasting evaluation
- `sample_submission.csv` — Example submission format
- `transactions.csv` — Daily transaction counts

For now, we will focus on with the most important file: **train.csv** and **test.csv**. Additional datasets can be merged depending on how you want to expand your model capabilities.

Loading the Dataset

```
import pandas as pd

file_path = "train.csv"
df = pd.read_csv(file_path)
```

Selecting Features and Target

Our objective is to predict sales. Therefore:

```
features = ["date", "store_nbr", "family", "onpromotion"]  
target = "sales"
```

The selected features represent:

- `date` → When the sale occurred
- `store_nbr` → Which store generated the sale
- `family` → Product category
- `onpromotion` → Number of products under promotion

The target variable is: `sales` , which represents the total number of items sold.

Dataset Overview

Let us first inspect the overall time span.

```
print("From", df["date"].min(), "to", df["date"].max())
```

From 2013-01-01 to 2017-08-15

Next, we examine the diversity of stores and product categories.

```
print("Number of stores:", df["store_nbr"].unique())  
print("Number of product families:", df["family"].unique())
```

Number of stores: 54

Number of product families: 33

Average Sales by Product Family

One simple question we can ask is: ***Which product categories sell the most?***

```
print(df.groupby("family")["sales"].mean().sort_values(ascending=False).head(10))
```

```
family
GROCERY I    4361.545269
BEVERAGES    2872.606897
PRODUCE      1945.159151
CLEANING     1170.986622
DAIRY         832.430034
BREAD/BAKERY  514.155229
MEATS         392.370327
POULTRY       383.773130
DELI          302.893000
PERSONAL CARE 292.296820
Name: sales, dtype: float64
```

Several interesting observations immediately emerge.

- Beverage products dominate sales volume.
- Cleaning supplies and dairy products are also highly demanded.
- Categories such as books and baby care contribute only a small fraction of total sales.

This imbalance is expected in retail environments, where essential consumer goods generate much larger sales volumes than specialized products.

Average Sales by Store and Product Family

Next, we investigate which combinations of stores and product categories generate the highest sales.

```

result = (df.groupby(["store_nbr", "family"]).agg({"sales": "mean"}).sort_values("sales",

ascending=False))
print(result.head(10))

```

	store_nbr	family	sales
45	GROCERY	I	12197.000000
44	GROCERY	I	11020.636364
47	GROCERY	I	10429.000000
44	BEVERAGES		9865.000000
	PRODUCE		9771.408500
46	GROCERY	I	9515.000000
49	GROCERY	I	9461.333333
3	BEVERAGES		9409.000000
48	GROCERY	I	9064.000000
3	GROCERY	I	8130.800000

The result reveals a clear pattern. The highest-selling combinations are dominated by large stores, grocery products, and beverage categories. This is not particularly surprising. Grocery products represent everyday necessities, and customers purchase them regularly regardless of season. For a forecasting model, this information is valuable because it suggests that product family and store identity are likely to be strong predictors of future sales.

And this brings us to the next stage of the pipeline. Before an RNN can learn temporal patterns, the raw data must first be transformed into a format suitable for sequential learning. Dates must be encoded, categorical values must be converted into numerical representations, and observations must be arranged into sequences that the network can process over time.

2.2. Prepare the Data

Before we can train an RNN, we must first prepare the data. This step is often underestimated by beginners. Many people imagine that Machine Learning is simply:

Load data → Train model → Get results

In reality, a large portion of the work happens before the model ever sees a single sample. Neural networks are particularly sensitive to data quality. Missing values, inconsistent formats, duplicate records, or poorly scaled features can negatively impact learning. Therefore, our first responsibility is to ensure that the dataset is clean and suitable for sequential modeling.

Data Quality Inspection

We begin by checking whether the dataset contains missing values or duplicate records.

```
# Check missing values
print('Number of missing values:', df.isnull().sum().sum())
# Check duplicated rows
print('Number of duplicated rows:', df.duplicated().sum())
```

```
Number of missing values: 0
Number of duplicated rows: 0
```

This is good news. The sampled dataset does not contain any missing observations or duplicated records. Therefore, no cleaning is required at this stage.

Convert and Sort Dates

Since we are working with a time-series forecasting problem, chronological order is extremely important. Unlike image classification, where individual samples can usually be shuffled freely, sequential models depend on the temporal relationship between observations.

For this reason, we convert the date column into a proper `datetime` format and sort the dataset in ascending order.

```
# Convert date column to datetime
df['date'] = pd.to_datetime(df['date'])
# Sort chronologically
df = df.sort_values('date')
```

This ensures that earlier observations always appear before later ones.

Checking Sales Values

Next, we verify whether the target variable contains invalid values.

```
print('Number of negative sales values:', (df['sales'] < 0).sum())
print('Number of missing sales values:', df['sales'].isnull().sum())
```

```
Number of negative sales values: 0
Number of missing sales values: 0
```

We also inspect the columns used in the forecasting model.

```
print('Missing values in selected columns:')
print(df.isnull().sum())
```

Missing values in selected columns:

```
date      0
store_nbr  0
family    0
sales     0
onpromotion 0
dtype: int64
```

All required features are complete and ready for further processing.

Splitting Data by Time

One of the most common mistakes in time-series forecasting is randomly shuffling the data before splitting it.

For ordinary Machine Learning problems, random shuffling is usually acceptable because observations are assumed to be independent.

Time-series data is different. Imagine training a model on sales from 2017 and then testing it on sales from 2014. From a forecasting perspective, this would be unrealistic because the model would effectively be learning from the future.

To avoid this problem, we split the dataset chronologically. The earliest observations become the training set, followed by validation data, while the newest observations are reserved for testing.

```
def split_data(df_input, train_ratio=0.7, val_ratio=0.15):
    train_size = int(len(df_input) * train_ratio)
    val_size = int(len(df_input) * val_ratio)

    df_train = df_input.iloc[:train_size].copy()
    df_val = df_input.iloc[train_size: train_size + val_size].copy()

    df_test = df_input.iloc[train_size + val_size:].copy()

    print('+ Number of training samples:', len(df_train))
    print('+ Number of validation samples:', len(df_val))
    print('+ Number of testing samples:', len(df_test))

    print('\nTrain:', df_train['date'].min(), '->', df_train['date'].max())
```

```

print('Validation:', df_val['date'].min(), '->', df_val['date'].max())
print('Test:', df_test['date'].min(), '->', df_test['date'].max())

return df_train, df_val, df_test

```

This split allows the model to learn from the past while being evaluated on future observations.

Scaling Numerical Features

Neural networks generally perform better when numerical values are standardized. Consider two variables:

- Sales values reaching thousands of units
- Promotion counts reaching only a few dozen

Without scaling, large numerical ranges can dominate the learning process. To address this issue, we apply Standard Scaling to the target variable.

Standardization transforms the data so that: $z = \frac{x-\mu}{\sigma}$, where: x is the original value, μ is the mean, σ is the standard deviation. The transformed values have: Mean ≈ 0 and Standard deviation ≈ 1

```

from sklearn.preprocessing import StandardScaler

def scale_data(df_train_input, df_val_input, df_test_input):
    scaler = StandardScaler()

    df_train_scaled = df_train_input.copy()
    df_val_scaled = df_val_input.copy()
    df_test_scaled = df_test_input.copy()

    df_train_scaled['sales'] = (scaler.fit_transform(df_train_scaled[['sales']]))
    df_val_scaled['sales'] = (scaler.transform(df_val_scaled[['sales']]))
    df_test_scaled['sales'] = (scaler.transform(df_test_scaled[['sales']]))

    return df_train_scaled, df_val_scaled, df_test_scaled, scaler

```

Here we only scale the 'sales' feature since we will apply sliding window in later step. The better practice is to apply scaling to every numerical feature found.

Notice that the scaler is fitted only on the training data. The validation and testing sets are transformed using the same parameters. This prevents information leakage from future observations.

Creating Time Features

Dates contain valuable information that neural networks cannot directly interpret. For example:

- December often behaves differently from January.
- Weekends may have different sales patterns compared to weekdays.
- Seasonal trends may occur throughout the year.

To expose this information, we extract several calendar-based features.

```

def create_time_features(dataframe_input):
    df_processed = (dataframe_input.copy())

    df_processed['year'] = (df_processed['date'].dt.year)
    df_processed['month'] = (df_processed['date'].dt.month)
    df_processed['day'] = (df_processed['date'].dt.day)
    df_processed['weekday'] = (df_processed['date'].dt.weekday)

```

```
df_processed['dayofmonth'] = (df_processed['date'].dt.day)

df_processed = (df_processed.drop(columns=['date']))
return df_processed
```

These features help the model identify recurring seasonal and weekly patterns.

Creating Lag Features

One of the most important concepts in forecasting is the idea that the past often influences the future. A store that sold a large quantity yesterday may also sell a large quantity today.

To capture this behavior, we create lag features. A lag feature simply stores previous sales values. For example:

- `sales_lag_1` → sales one day ago
- `sales_lag_7` → sales seven days ago
- `sales_lag_14` → sales fourteen days ago

```
def create_lag_features(dataframe_input):
    df_processed = (dataframe_input.copy())

    for i in range(1, 15):
        df_processed[f'sales_lag_{i}'] = (df_processed['sales'].shift(i).fillna(0))

    return df_processed
```

Lag features provide explicit memory about previous observations, which is particularly valuable for forecasting models.

Creating Rolling Mean Features

Individual sales values can be noisy. One unusually large order may create a spike that does not represent the overall trend.

To smooth these fluctuations, we create rolling averages. A rolling mean computes the average sales over a moving window. Examples:

- Average sales over the previous 7 days
- Average sales over the previous 14 days

```
def create_rolling_mean_features(dataframe_input):
    df_processed = (dataframe_input.copy())

    for i in [7, 14]:
        df_processed[f'sales_rolling_mean_{i}'] = (df_processed['sales'].rolling(window=i).mean().fillna(0))

    return df_processed
```

These features help reveal longer-term trends while reducing short-term noise.

Encoding Categorical Variables

The dataset contains categorical information such as store number or product family.

Neural networks cannot directly process text labels. Therefore, these categories must be converted into numerical representations. We use One-Hot Encoding.

For example:

Family	Family_BEVERAGES	Family_DAIRY	Family_BREAD
BEVERAGES	1	0	0
DAIRY	0	1	0
BREAD	0	0	1

This representation prevents the model from assuming that categories have a natural numerical order.

```
from sklearn.preprocessing import OneHotEncoder

def one_hot_encode_and_concat(df_train_input, df_val_input,
                              df_test_input, categorical_features):

    encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')

    train_encoded = encoder.fit_transform(df_train_input[categorical_features])
    val_encoded = encoder.transform(df_val_input[categorical_features])
    test_encoded = encoder.transform(df_test_input[categorical_features])
```

```

    encoded_names = (encoder.get_feature_names_out(categorical_features))

    df_train_encoded = pd.DataFrame(train_encoded, columns=encoded_names,
                                     index=df_train_input.index)
    df_val_encoded = pd.DataFrame(val_encoded, columns=encoded_names,
                                   index=df_val_input.index)
    df_test_encoded = pd.DataFrame(test_encoded, columns=encoded_names,
                                    index=df_test_input.index)

    df_train_processed = pd.concat([df_train_input.drop(columns=categorical_features),
                                    df_train_encoded], axis=1)
    df_val_processed = pd.concat([df_val_input.drop(columns=categorical_features),
                                   df_val_encoded], axis=1)
    df_test_processed = pd.concat([df_test_input.drop(columns=categorical_features),
                                    df_test_encoded], axis=1)

    print("Shape of train:", df_train_processed.shape)
    print("Shape of validation:", df_val_processed.shape)
    print("Shape of test:", df_test_processed.shape)

    return df_train_processed, df_val_processed, df_test_processed, encoder

```

At this point, the dataset has been cleaned, scaled, enriched with temporal information, enhanced with lag and rolling statistics, and transformed into a numerical format suitable for neural networks.

2.3. Visualizing the Data

Before building any forecasting model, there is an important question we should ask:
What does the data actually look like?

In many Machine Learning projects, visualization is helpful. In time-series forecasting, visualization is almost mandatory.

Unlike tabular datasets where rows are often independent, time-series data contains trends, seasonality, cycles, and temporal relationships that are difficult to understand by looking at numbers alone.

A few simple plots can often reveal patterns that significantly influence model design.

Sales Trend Over Time

We begin by visualizing sales throughout the training period.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Temporal data splitting
train_size = int(len(df) * 0.7)
val_size = int(len(df) * 0.15)

# Temporary dataframes for visualization
df_plot_train = df.iloc[:train_size].copy()
df_plot_val = df.iloc[train_size:train_size + val_size].copy()
df_plot_test = df.iloc[train_size + val_size:].copy()

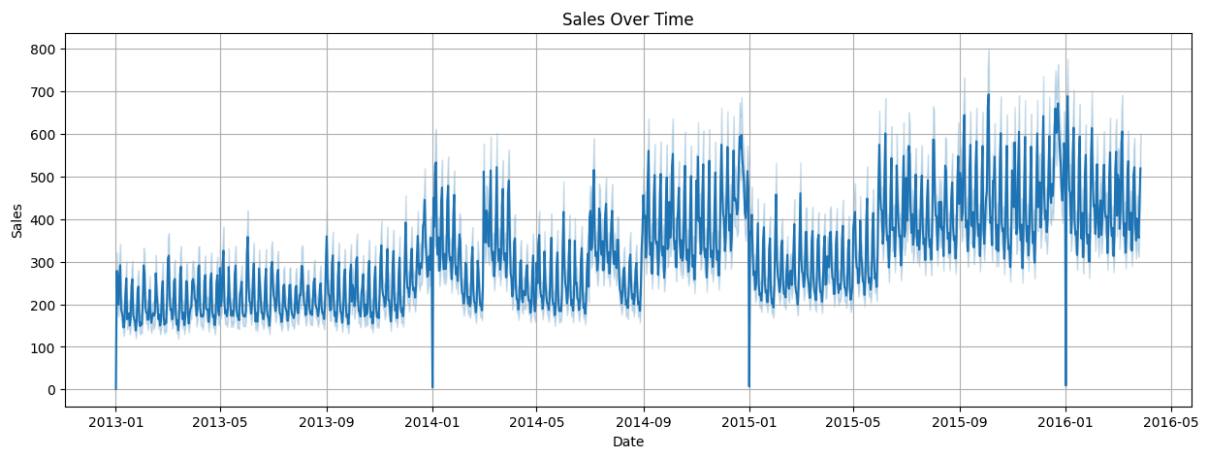
# Ensure datetime format
df_plot_train['date'] = pd.to_datetime(df_plot_train['date'])
df_plot_val['date'] = pd.to_datetime(df_plot_val['date'])
df_plot_test['date'] = pd.to_datetime(df_plot_test['date'])

plt.figure(figsize=(15, 5))

sns.lineplot(x='date', y='sales', data=df_plot_train)

plt.title('Sales Over Time')
plt.xlabel('Date')
plt.ylabel('Sales')
plt.grid(True)

plt.show()
```

The chart reveals a noticeable upward trend in sales over time. We can also observe several large spikes occurring near the end of each year. These spikes may correspond to major shopping seasons, holiday periods, or promotional campaigns.

This is a common characteristic of retail sales data. Consumer spending rarely remains constant throughout the year.

Monthly Seasonality

To investigate seasonal behavior more closely, we extract several calendar-related features.

```
df_plot_train['year'] = df_plot_train['date'].dt.year
df_plot_train['month'] = df_plot_train['date'].dt.month
df_plot_train['weekday'] = df_plot_train['date'].dt.weekday
df_plot_train['dayofweek_name'] = df_plot_train['date'].dt.day_name()
```

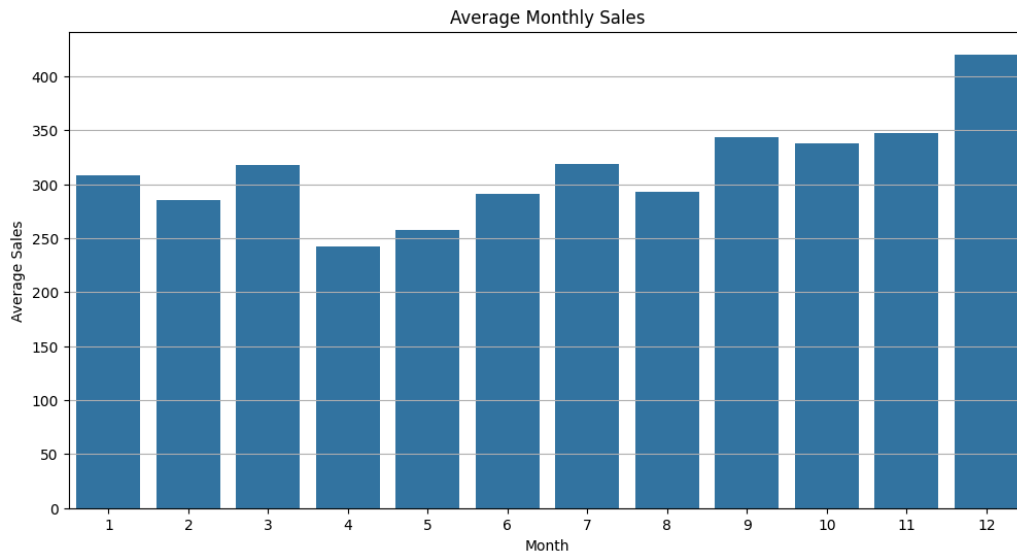
Next, we compute the average sales for each month.

```
plt.figure(figsize=(12, 6))

sns.barplot(x='month', y='sales',
            data=df_plot_train.groupby('month')['sales'].mean().reset_index())

plt.title('Average Monthly Sales')
plt.xlabel('Month')
plt.ylabel('Average Sales')
```

```
plt.grid(axis='y')
plt.show()
```



The monthly pattern suggests that sales gradually increase toward the end of the year. December often exhibits the highest sales volume, which aligns with typical retail behavior during major holiday shopping periods.

This seasonal pattern is exactly the type of information an RNN should learn when forecasting future sales.

Weekly Seasonality

Consumer behavior also changes throughout the week. To explore this effect, we compare average sales across weekdays.

```
weekday_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday',
                  'Friday', 'Saturday', 'Sunday']

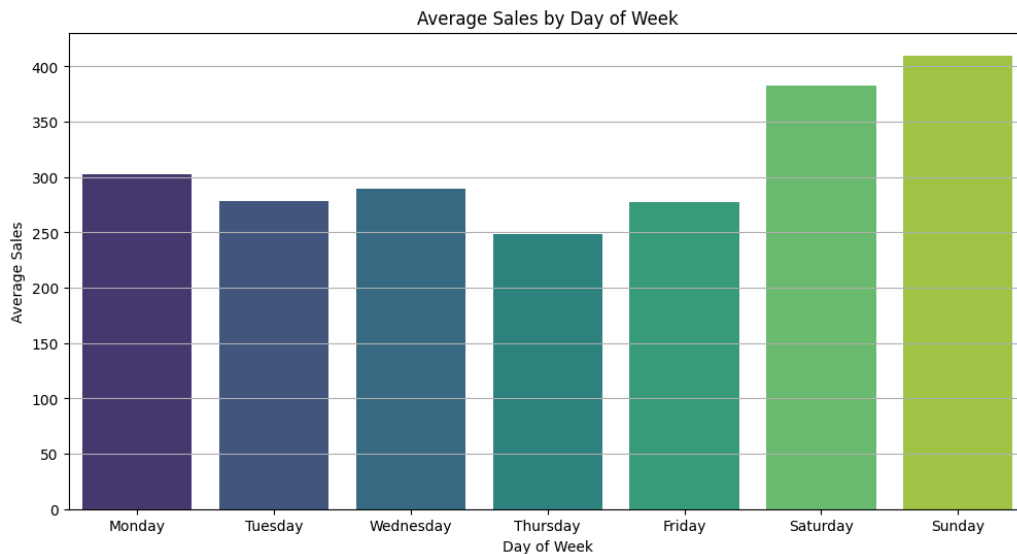
df_weekday_sales = (df_plot_train.groupby('dayofweek_name')['sales']
                    .mean().reindex(weekday_order).reset_index())

plt.figure(figsize=(12, 6))

sns.barplot(x='dayofweek_name', y='sales', data=df_weekday_sales,
            palette='viridis')
```

```
plt.title('Average Sales by Day of Week')
plt.xlabel('Day of Week')
plt.ylabel('Average Sales')

plt.grid(axis='y')
plt.show()
```



Sales tend to be higher during weekends, particularly on Saturday and Sunday. Early weekdays generally experience lower sales volumes.

This pattern makes intuitive sense. Customers often have more free time to shop during weekends, resulting in increased retail activity.

Impact of Promotions

One of the most important business questions is: **Do promotions actually increase sales?** We begin with a simple comparison.

```

promo_sales = (df_plot_train[df_plot_train['onpromotion'] > 0]['sales'].mean())
no_promo_sales = (df_plot_train[df_plot_train['onpromotion'] == 0]['sales'].mean())

print(f"Average sales with promotion: {promo_sales:.2f}")
print(f"Average sales without promotion: {no_promo_sales:.2f}")

```

Average sales with promotion: 1240.01

Average sales without promotion: 111.09

To obtain a more detailed view, we examine promotion effects across product families.

```

df_plot_train['is_on_promotion'] = (df_plot_train['onpromotion'] > 0)

promo_effect_per_family = (df_plot_train.groupby(['family', 'is_on_promotion'])['sales'].mean().reset_index())

plt.figure(figsize=(18, 8))

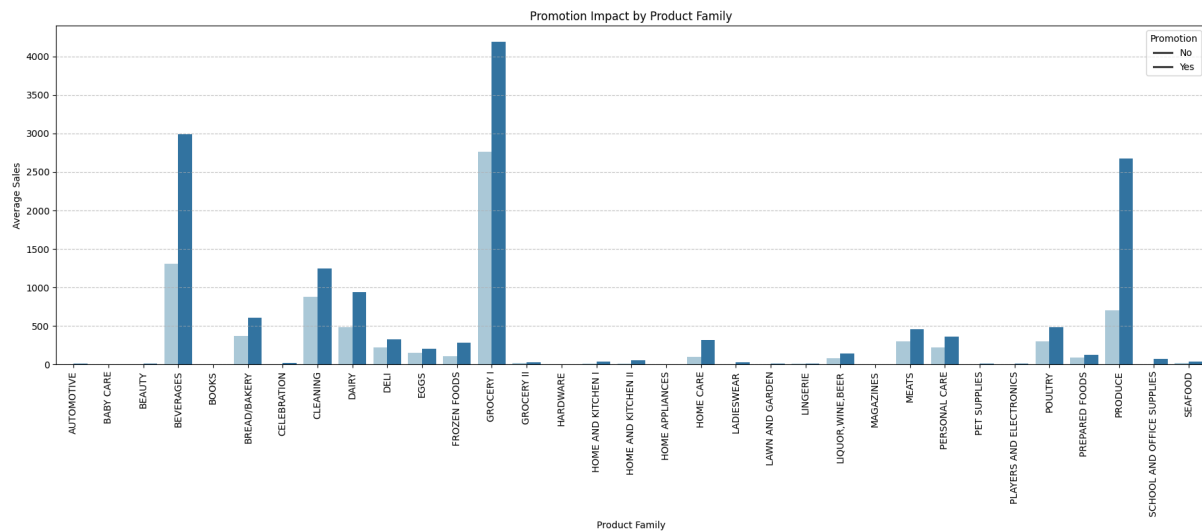
sns.barplot(x='family', y='sales', hue='is_on_promotion', data=promo_effect_per_family, palette='Paired')

plt.title('Promotion Impact by Product Family')
plt.xlabel('Product Family')
plt.ylabel('Average Sales')

plt.xticks(rotation=90)
plt.legend(title='Promotion', labels=['No', 'Yes'])
plt.grid(axis='y', linestyle='--', alpha=0.7)

plt.tight_layout()
plt.show()

```



Promotions generally increase sales across most product categories. However, the magnitude of the effect varies considerably.

Product families such as: BEVERAGES; CLEANING; DAIRY; GROCERY I; PRODUCE show particularly strong responses to promotional campaigns. This suggests that promotional activity is likely to be an important predictor for future sales forecasting.

2.4. Building the Complete Preprocessing Pipeline

After merging, we can finally assemble all preprocessing steps into a single workflow. The pipeline performs the following operations:

1. Sort data chronologically.
2. Split data into train, validation, and test sets.
3. Create time-based features.
4. Create lag features.
5. Create rolling averages.
6. Apply Scaling and One-Hot Encoding.

The result is a fully processed dataset ready for sequential learning.

```

df['date'] = pd.to_datetime(df['date'])
df = (df.sort_values('date').reset_index(drop=True))

df_train, df_val, df_test = split_data(df, train_ratio=0.7, val_ratio=0.15)

df_train = create_time_features(df_train)
df_val = create_time_features(df_val)
df_test = create_time_features(df_test)

df_train = create_lag_features(df_train)
df_val = create_lag_features(df_val)
df_test = create_lag_features(df_test)

df_train = create_rolling_mean_features(df_train)
df_val = create_rolling_mean_features(df_val)
df_test = create_rolling_mean_features(df_test)

categorical_features = ['store_nbr', 'family']

df_train, df_val, df_test, one_hot_encoder = (one_hot_encode_and_concat(df_train, df_val,
df_test, categorical_features))
numerical_features_to_scale = [
    'sales', 'onpromotion', 'year', 'month', 'day', 'weekday', 'dayofmonth',
    'sales_lag_1', 'sales_lag_2', 'sales_lag_3', 'sales_lag_4', 'sales_lag_5',
    'sales_lag_6', 'sales_lag_7', 'sales_lag_8', 'sales_lag_9', 'sales_lag_10',
    'sales_lag_11', 'sales_lag_12', 'sales_lag_13', 'sales_lag_14',
    'sales_rolling_mean_7', 'sales_rolling_mean_14'
]
df_train, df_val, df_test, main_scaler = scale_data(df_train, df_val, df_test,
numerical_features_to_scale)

```

Key Outputs:

- Number of training samples: 2100621
- Number of validation samples: 450133
- Number of testing samples: 450134

Train: 2013-01-01 00:00:00 → 2016-03-27 00:00:00

Validation: 2016-03-27 00:00:00 → 2016-12-05 00:00:00

Test: 2016-12-05 00:00:00 → 2017-08-15 00:00:00

df_val shape: (450133, 110)
df_test shape: (450134, 110)

At the end of this stage, the raw retail dataset has been transformed into a structured machine learning dataset containing historical, temporal, business, promotional, and economic information.

The final step before training is to convert these observations into sequences that an RNN can understand.

2.5. Sliding Window

Recurrent Neural Networks, LSTMs, and GRUs are designed to learn from **sequences**, not individual observations. While our dataset currently exists as a table where each row represents a single day, recurrent models expect each training example to contain a history of observations that occurred over multiple timesteps.

This means we must transform our tabular dataset into a collection of sequences. The most common technique for accomplishing this is called the **Sliding Window**.

The basic idea is straightforward:

- Select a fixed number of consecutive observations.
- Use those observations as the model input.
- Use the following observation as the prediction target.
- Move the window forward by one step.
- Repeat until the end of the dataset is reached.

Suppose we have the following simplified data:

Day	Sales	Promotion
1	10	0
2	12	1
3	15	0
4	18	1
5	20	0
6	25	1
7	30	0

If we choose a window size of **3**, the generated sequences become:

Input Sequence (X)	Target (y)
[[10,0], [12,1], [15,0]]	18
[[12,1], [15,0], [18,1]]	20
[[15,0], [18,1], [20,0]]	25
[[18,1], [20,0], [25,1]]	30

Notice that each training sample contains multiple timesteps rather than a single observation. The model can therefore learn temporal relationships such as trends, seasonality, and the effects of promotions over time.

Because the window maintains a fixed size while continuously moving forward through the sequence, the technique is known as a **sliding window**.

Creating Sliding Windows with TensorFlow

Sliding windows can be created manually using tensors. TensorFlow provides a convenient and efficient approach through the function `timeseries_dataset_from_array()`.

This function automatically converts a continuous sequence into batches of sliding windows suitable for recurrent neural networks.

For this experiment, we choose: `window_size = 60` and `batch_size = 64`. The model will therefore observe the previous **60 days** when attempting to predict sales for the next day.

Before creating the datasets, we separate the input features from the target variable.

```
train_features = df_train[selected_features].values
train_target = df_train['sales'].values

val_features = df_val[selected_features].values
val_target = df_val['sales'].values

test_features = df_test[selected_features].values
test_target = df_test['sales'].values
```

Next, TensorFlow automatically generates the sliding windows.

```
train_dataset = tf.keras.utils.timeseries_dataset_from_array(
    data=train_features,
    targets=train_target[window_size:],
    sequence_length=window_size,
    sampling_rate=1,
    stride=1,
    sequence_stride=False,
```



```

        batch_size=batch_size
    )

    val_dataset = tf.keras.utils.timeseries_dataset_from_array(
        data=val_features,
        targets=val_target[window_size:],
        sequence_length=window_size,
        sampling_rate=1,
        sequence_stride=1,
        shuffle=False,
        batch_size=batch_size
    )

    test_dataset = tf.keras.utils.timeseries_dataset_from_array(
        data=test_features,
        targets=test_target[window_size:],
        sequence_length=window_size,
        sampling_rate=1,
        sequence_stride=1,
        shuffle=False,
        batch_size=batch_size
    )

```

The parameter choices are important:

- `sequence_length=60` means each input sequence contains sixty consecutive days.
- `stride=1` moves the window forward one day at a time.
- `sampling_rate=1` uses every observation.
- `shuffle=False` preserves chronological order, which is critical for time-series data.
- `batch_size=64` groups multiple sequences together during training.

To verify the dataset structure, we can inspect a single batch.

```

for batch_x, batch_y in train_dataset.take(1):
    print(f"X_train batch shape: {batch_x.shape}")
    print(f"y_train batch shape: {batch_y.shape}")

```

The output follows the form:

X_train batch shape:
(64, 60, num_features)

y_train batch shape:
(64,)

This structure is exactly what recurrent neural networks expect:

- **64 sequences** per batch.
- **60 timesteps** per sequence.
- **num_features** variables observed at each timestep.

Finally, we can inspect the total number of batches.

```
print(f"Train batches: {len(train_dataset)}")
print(f"Validation batches: {len(val_dataset)}")
print(f"Test batches: {len(test_dataset)}")
```

Train batches: 32822

Validation batches: 7033

Test batches: 7033

The original store-sales dataset contains approximately **3 million records**, and in this experiment we use the entire dataset. However, once the data is transformed into **60-day sliding windows**, the model no longer treats each row as an independent training example. Instead, every sample becomes a sequence of **60 consecutive observations** used to predict the next day's sales.

These sequences are then grouped into batches of **64 samples** using TensorFlow's Dataset API, resulting in:

- **32,822 training batches**
- **7,033 validation batches**
- **7,033 testing batches**

Therefore, although the underlying dataset still contains millions of rows, the model learns from thousands of mini-batches of sequential data. Each sequence provides sixty days of historical context, allowing recurrent networks to capture temporal patterns that would be invisible if the observations were treated independently.

Why Use a 60-Day Window?

The choice of window size is one of the most important design decisions in time-series forecasting.

A very short window may fail to capture meaningful temporal patterns, while an excessively large window increases computational cost and may introduce irrelevant historical information.

For retail sales forecasting, a sixty-day window provides several advantages:

- Captures short-term sales trends.

- Captures multiple weekly cycles.
- Captures recurring weekend effects.
- Captures the influence of recent promotions.
- Provides enough historical context for LSTM and GRU models to learn longer-term dependencies.

While there is no universally optimal window size, sixty days offers a reasonable balance between historical context and computational efficiency for this dataset.

After applying the sliding window transformation, the dataset is finally in a form suitable for recurrent neural networks. Instead of observing isolated days, the model now learns from sixty-day sequences, allowing it to discover how historical sales patterns influence future demand.

2.6 Training a Recurrent Neural Network

With the sliding-window datasets prepared, we can now train our first recurrent neural network.

Unlike the earlier implementation where we manually reshaped NumPy arrays, the TensorFlow datasets already provide sequences in the correct format:

```
(batch_size, window_size, num_features)
```

This is exactly the structure expected by recurrent layers such as SimpleRNN, LSTM, and GRU.

Building the RNN Model

For this experiment, we use a stacked SimpleRNN architecture consisting of:

- Two recurrent layers
- One fully connected hidden layer
- One dropout layer
- One output neuron for sales prediction

```
from tensorflow import keras

# Determine number of input features automatically
num_features = len(selected_features)

rnn_model = keras.Sequential([
```

```

keras.layers.SimpleRNN(64, return_sequences=True, input_shape=
(window_size, num_features)),
keras.layers.SimpleRNN(64),
keras.layers.Dense(128, activation='relu'),
keras.layers.Dropout(0.4),
keras.layers.Dense(1)
])

rnn_model.compile(optimizer='adam', loss='mean_squared_error', met
rics=['mae'])
rnn_model.summary()

```

Because this is a regression problem, the output layer contains a single neuron and no activation function.

Training the Model

To reduce overfitting, Early Stopping is applied.

```

from tensorflow.keras.callbacks import EarlyStopping

early_stopping_rnn = EarlyStopping(monitor='val_loss',
                                   patience=2, restore_best_weight
s=True)

history_rnn = rnn_model.fit(train_dataset, validation_data=val_dat
aset,
                             epochs=5, callbacks=[early_stopping_rn
n])

```

We do not need to specify `batch_size` during model training because it was already defined when creating the TensorFlow datasets using `timeseries_dataset_from_array()`.

Each dataset already contains pre-batched sequences, so the model automatically receives batches of size 64 during training. In other words, batching is handled by the dataset itself rather than by the `fit()` function.

Visualizing Training Loss

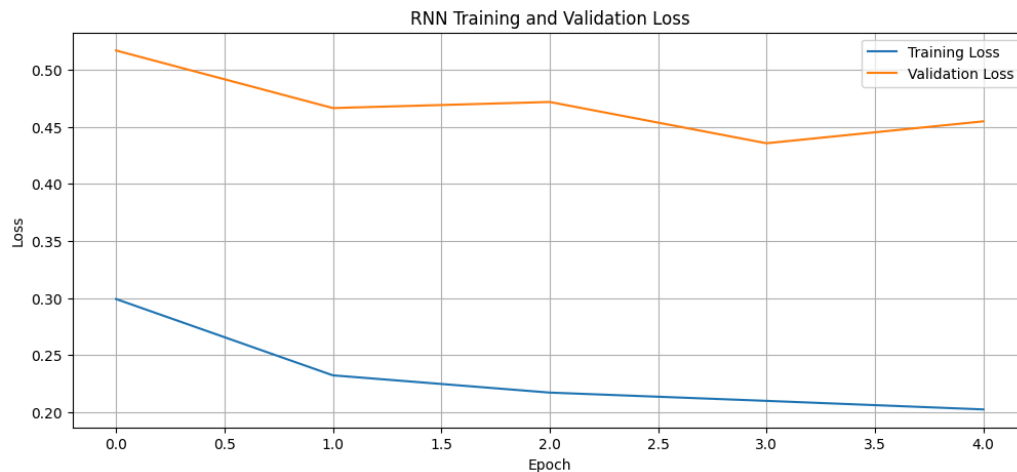
```

import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
plt.plot(history_rnn.history['loss'], label='Training Loss')

```

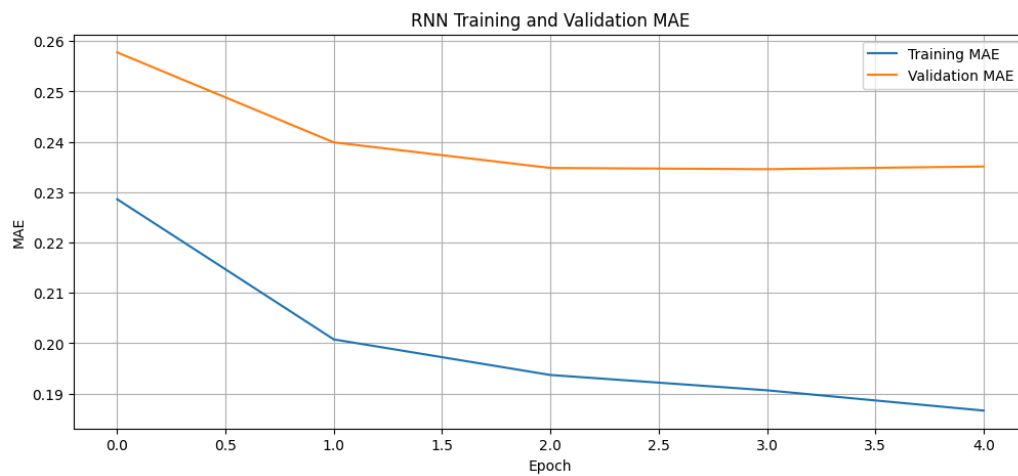
```
plt.plot(history_rnn.history['val_loss'], label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
plt.show()
```



A decreasing training loss together with a decreasing validation loss indicates that the model is successfully learning useful patterns from the sales history.

Visualizing MAE

```
plt.figure(figsize=(10, 6))
plt.plot(history_rnn.history['mae'], label='Training MAE')
plt.plot(history_rnn.history['val_mae'], label='Validation MAE')
plt.title('Training and Validation MAE')
plt.xlabel('Epoch')
plt.ylabel('MAE')
plt.legend()
plt.grid(True)
plt.show()
```



The Mean Absolute Error (MAE) is often easier to interpret because it represents the average prediction error in the same units as the target variable.

Evaluating the Model

Once training is complete, predictions can be generated for the testing dataset.

```

from sklearn.metrics import mean_absolute_error, mean_squared_error

y_pred_scaled_rnn = rnn_model.predict(test_dataset)

# Collect actual targets from tf.data.Dataset
y_test_scaled = np.concatenate([y.numpy() for _, y in test_dataset])

# Convert back to original sales values
y_test_actual = main_scaler.inverse_transform(y_test_scaled.reshape(-1, 1))
y_pred_actual = main_scaler.inverse_transform(y_pred_scaled_rnn)

mae_rnn = mean_absolute_error(y_test_actual, y_pred_actual)
mse_rnn = mean_squared_error(y_test_actual, y_pred_actual)
rmse_rnn = np.sqrt(mse_rnn)

print(f"Test MAE: {mae_rnn:.2f}")
print(f"Test MSE: {mse_rnn:.2f}")
print(f"Test RMSE: {rmse_rnn:.2f}")

```

Test MAE: 86.99

Test MSE: 50839.07

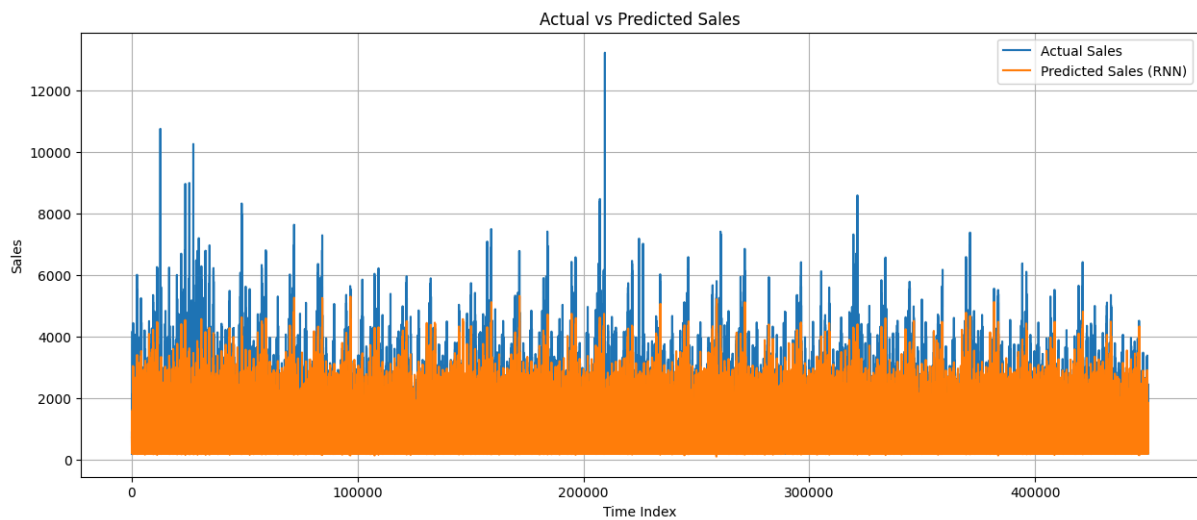
Test RMSE: 225.48

Actual vs Predicted Sales

```

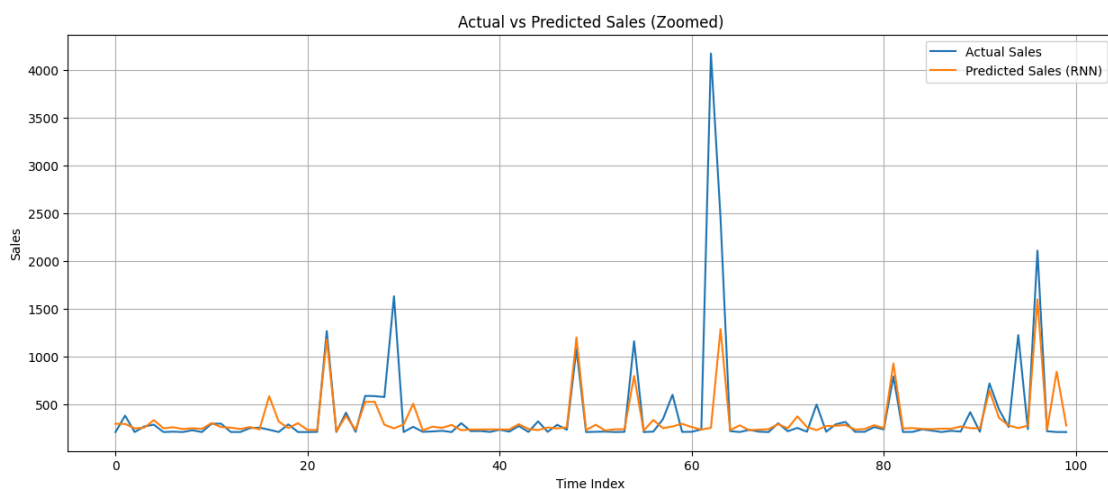
plt.figure(figsize=(15, 6))
plt.plot(y_test_actual, label='Actual Sales')
plt.plot(y_pred_actual, label='Predicted Sales')
plt.title('Actual vs Predicted Sales')
plt.xlabel('Time Index')
plt.ylabel('Sales')
plt.legend()
plt.grid(True)
plt.show()

```



To inspect local behavior more closely, we can zoom into a smaller region of the testing period.

```
plt.figure(figsize=(15, 6))
plt.plot(y_test_actual[200:300], label='Actual Sales')
plt.plot(y_pred_actual[200:300], label='Predicted Sales')
plt.title('Actual vs Predicted Sales (Zoomed)')
plt.xlabel('Time Index')
plt.ylabel('Sales')
plt.legend()
plt.grid(True)
plt.show()
```



The resulting plots allow us to visually evaluate how well the model captures trends, seasonal effects, and short-term fluctuations in store sales.

While SimpleRNN introduces the concept of temporal memory, it often struggles to retain information over longer sequences. In the next sections, we will revisit the same forecasting problem using **LSTM** and **GRU** architectures to see how gated recurrent networks improve upon the limitations of traditional RNNs.

This experiment demonstrates how a Recurrent Neural Network can learn temporal relationships from historical sales data. Unlike traditional feed-forward neural networks, the RNN maintains an internal hidden state that allows information from previous timesteps to influence future predictions. This capability makes it particularly suitable for sequential forecasting problems.

*However, SimpleRNN architectures have an important limitation. As sequences become longer, the network struggles to preserve information from distant timesteps. This issue is known as the **vanishing gradient problem**, and it limits the ability of standard RNNs to capture long-term dependencies.*

*To address this weakness, researchers developed more advanced recurrent architectures. The most famous of these is the **Long Short-Term Memory (LSTM)** network.*

In the next section, we will revisit the same Store Sales dataset and replace the SimpleRNN architecture with LSTM in order to explore how memory gates allow the network to retain information over much longer periods of time.

Summary

In this section, we built our first Recurrent Neural Network for time-series forecasting using the Store Sales dataset. After transforming the data into sliding-window sequences, we trained a SimpleRNN model to predict future sales based on previous observations.

The model successfully demonstrated how recurrent connections enable neural networks to learn temporal patterns, making RNNs fundamentally different from traditional feed-forward architectures.

Although the model was able to capture short-term trends, its limited memory makes it less effective for long sequences, motivating the need for more advanced architectures such as LSTM, which will be introduced in the next section.

References

<https://www.geeksforgeeks.org/machine-learning/introduction-to-recurrent-neural-network/>

<https://www.youtube.com/watch?v=AsNTP8Kwu80&themeRefresh=1>

<https://www.ibm.com/think/topics/recurrent-neural-networks>

Next Section:

 [16.1. Long Short-Term Memory Networks \(LSTM\)](#)